Busy Developer's Guide to Python

Ted Neward

Neward & Associates

ted.neward@newardassociates.com
http://blogs.newardassociates.com

Objectives

In this talk, we shall...

- Talk about how to get started with Python
- Go over some of the key syntax and semantics
- Explore just tiny bits of the Python universe

Why... do you care, again

Because...

- Data Science
- Machine Learning
- AI
- *(repeat it to yourself until you accept it)*

... driving the operating system

SCRIPTING WITH PYTHON

Scripting with Python

Python has great OS-level packages

- Windows (Win32)
  - pywin32 (https://pypi.org/project/pywin32/)
  - pyinstaller (https://pypi.org/project/pyinstaller/)
- macOS
  - PyObjC (https://pypi.org/project/pyobjc/)
  - py2app (http://undefined.org/python/#py2app)

Scripting with Python

An annoying example

```
import win32api

# Call ::Beep()
win32api.Beep(500, 3000)

# Call ::GetSystemMetrics to get screen dimensions
print('Width: ', win32api.GetSystemMetrics(0))
```

Traditional desktop applications

PYTHON GUIS

Python GUIs

A few options

- Tk / Tcl-Tk / tkinter
  - Back in the day, Tcl (language) had a widget toolkit (Tk)

    This became a popular cross-platform GUI toolkit
  - Python build bindings to the Tk framework

    This became known as "tkinter"
  - Full Python docs:

    https://docs.python.org/3/library/tk.html
- (wrappers around) Qt
- (wrappers around) wxWidgets

Building a simple desktop game

PYGAME

PyGame

Pygame: Game engine for Python

- Python wrapper around SDL
  - Simple DirectMedia Layer
  - cross-platform
  - access to hardware (sound, video, input)
- `pip3 install pygame`
  - `python -m pygame.examples.aliens`
- *NOTE*: this is not a 3D game engine
  - Don't expect to take sales away from Baldur's Gate 3 with this

An Overview

PYTHON

Python

What is Python?

"Python is a programming language that lets you work quickly and integrate systems more effectively."- http://www.python.org

Python

What is Python?

"Python is an easy to learn, powerful programming language. It has efficient high-level data structures and a simple but effective approach to object-oriented programming. Python's elegant syntax and dynamic typing, together with its interpreted nature, make it an ideal language for scripting and rapid application development in many areas on most platforms."

Python

What is Python?

- (mostly) portable
- open-source
- extensive platform
- "scripting" emphasis

Getting the bits and stuff

GETTING STARTED

Getting Started

Python is an open-source language

- many different ways to obtain bits
  - get source and build
  - prepackaged distributions
  - often ships with other tools
- beware OS pre-installs!
  - these can be wildly out-of-date
  - (looking at you, Apple....)
- Python comes in two distinct flavors
  - Python2 (2.x) and Python3 (3.x)

Getting Started

Package managers

- Homebrew: `brew install python3`
- Debian/Ubuntu: `sudo apt install python3 python3-pip python3-venv`
- Windows: Microsoft Store (but this is considered unstable!)
- these may all be *slightly* behind latest-and-greatest
    - takes a while (a week?) to catch up to the newest

Getting Started

Python direct

- https://www.python.org/downloads/
- binary installers
  - supports most mainstream OSes (Linux, Windows, macOS)
  - usually integrates with the OS in some fashion (launchers, registered extensions, etc)
- third-party distributions are also available
  - ActiveState: https://www.activestate.com/products/python/

Getting Started

Build from source

- https://github.com/python/cpython
- https://www.python.org/downloads/source/
- Not recommended for the casual evaluator
- But highly recommended if you want to spelunk!
    - Python is ridiculously well-engineered

Getting Started

Python command line

- `python` or `python3`
- -h: Help
- --version: Display the version
- -i: Enter interactive mode after script execution
- -v: Print message on each module initialization

Getting Started

Python environment variables

- most of these are unnecessary for app development
- but still useful to know for a variety of edge cases
- `PYTHONHOME`

  alternative location to use for Python's install directory
- `PYTHONPATH`

  paths to search for modules

Getting Started

Python REPL

- everything is immediately available within the REPL
- exact same environment as executed code, just... interactive
- great way to get started working with Python for system tasks or experiments

Getting Started

Hello, Python

- fire up Python at the command-line
- `python` is typically a version-agnostic executable

  this *should* be Python3, but no guarantees
- `python3` is typically a Python3 version-agnostic executable
- `python3.12` is typically bound to that exact version of Python

Getting Started

Greetings at the the terminal

```
tedneward@Teds-MBP-16 External % python3
Python 3.12.5 (main, Aug  6 2024, 19:08:49) [Clang 15.0.0 (clang-1500.3.9.4)] on
darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> print('Hello world!')
Hello world!
>>>
```

Python workspace/execution context management

PYTHON ENVIRONMENTS

Python Environments

Python application execution

- Python is a "source-executed" language
- where and how do we keep sources?
- how do we manage different versions
  - across applications?
  - across libraries (dependencies)?

Python Environments

More detailed breakdown

- Environment management (which is mostly concerned with virtual environments)
- Package management
- Python version management
- Package building
- Package publishing

Python Environments

Python has numerous solutions here

- venv
- virtualenv
- pip
- pipenv
- poetry
- Anaconda/conda
- Full list/analysis at https://alpopkes.com/posts/python/packagin

A virtual environment for Python

VENV

venv

What is venv?

- Python package for creating virtual environments
- shipped with Python; does not have to be installed by the user

venv

Commands

- Create new environment:
  - `python3 -m venv` *env-name* : Creates new directory
  - `python3 -m venv .` : Creates virtual environment in curdir

venv

What happens?

- Directory structure created
  - `bin`: commands/executable (shell scripts) installed
  - `lib`: installed packages
  - `include`: necessary development (header) files
- Everything is localized via this file structure

venv

Using it

- Activate that environment:
  - `.` *`env-name`*`/bin/activate`
  - `source /bin/activate`
- The `bin` directory will now prepend the PATH
  - it will contain a localized reference to Python
  - ... and various utilities
  - ... and any new utilities installed
  - ... and so on
- Deactivate it: `deactivate`

Python's Package Manager

PIP

pip

Global packages

- Python packages stored at Python Package Index
  - https://pypi.org
- `pip` (or `pip3`) CLI for installation/management of packages
  - lately, convention is to run it as a module rather than through script
  - `python -m pip` *command-line-arguments*
- generally works like all other package

pip

Install

- from PyPI:
  - `python -m pip install sampleproject`
- from GitHub:
  - `python -m pip install git+https://github.com/pypa/sampleproject.git@main`
- from local distribution file:
  - source: `python -m pip install`

pip

Install from requirements

- `python -m pip install -r requirements.txt`
- convention dictates the filename (`requirements.txt`)
- file contents: a list of `pip install` arguments (package names)
- optional version guidelines/constraints
  - `pkg1<1.3`: Means anything earlier than 1.3
  - `pkg1>=1.0,<=2.0`: Anything between 1.0 and

pip

Other commands

- List: `python -m pip list`
- Upgrade: `python -m pip install --upgrade sampleproject`
- Uninstall/remove: `python -m pip uninstall sampleproject`

An Overview

PYTHON LANGUAGE

Python Language

At a high level

- object (class)-oriented
- dynamically-typed
- preferential to words over symbols
- moderately convention-based
  - significant whitespace
  - standard operator custom definitions
- continuous improvement
  - see "PEPs": https://peps.python.org/pep-0000/

Python Language

"The Zen of Python" (PEP 20)

- Beautiful is better than ugly.
- Explicit is better than implicit.
- Simple is better than complex.
- Complex is better than complicated.
- Flat is better than nested.
- Sparse is better than dense.
- Readability counts.
- Special cases aren't special enough to break

Python Language

"The Zen of Python"

- Errors should never pass silently.
- Unless explicitly silenced.
- In the face of ambiguity, refuse the temptation to guess.
- There should be one--and preferably only one--obvious way to do it.
- Although that way may not be obvious at first unless you're Dutch.
- Now is better than never.

Python Language

When in doubt....

```
import this
```

Syntax, comments, types, oh my!

BASICS

Basics

Source execution model

- often called "interpreted"
  - not entirely accurate
  - most Pythons parse to bytecode, then execute that
- either feed scripts to executable directly
  - arguments passed after script name
- or (on *nix) use "she-bang" style at top of script
- or use REPL directly to experiment

Basics

Hello, Python

```
#!/usr/bin/env python
```

```
# Say hello, Python
msg = "Hello, Python!"

print(msg)
```

Basics

Comments

- "#"-prefixed, run to end-of-line
- no multi-line comments at all
  - just use lots of "#"s

Basics

Variables

- introduced upon initialization
  - not on first reference; must be initialized first
- variable names can be any alphanumeric symbol
  - variables and functions are snake_cased
  - classes are CamelCased
- multiple declaration, assignment permitted

Basics

Variables

```
num = 5
message = "This is a string"
this_is_a_really_long_variable = 0
x,y = 0,0
#uninitialized_variable  # error!
```

Basics

Stylistic/Idiomatic

- use multiple assignment for identical initialization
  - this is not widespread
- use the "walrus operator" (`:=`) for assignment expressions
  - useful in complex initialization scenarios

Basics

Walrus assignment

```
assignment = (truth := True)
print(assignment)
```

Basics

Variable types

- `None`: Python's equivalent to `null`/`nil`
- numbers.Number: four subtypes
  - numbers.Integral (`int`): unlimited range
  - Boolean (`bool`) is a subtype of int; `True`/`False`
  - numbers.Real (`float`): machine-dependent range
  - numbers.Complex (`complex`): complex numbers

Basics

Variable types

- Sequences: finite ordered sets
  - Bytes (`bytes`): immutable array of bytes
  - Strings (`str`): Unicode
  - Tuples: arbitrary gathering of unnamed fields
- Objects (`object`): object references
  - Lists (`list`): ordered sequence of values
  - Dictionaries (`dict`): unordered collection of key-value pairings
  - Sets (`set`): unordered collection disallowing

Basics

Numbers

- all the usual mathematical operations
- `/` always yields floating-point division
- `//` always yields integral ("floor rounding") division
- `**` exponentiation operator

Basics

Numbers

```
div1 = 5 / 2      # 2.5
div2 = 5 // 2     # 2
square = 2 ** 2   # 4
```

Basics

Strings

- single-quoted or double-quoted
- `r`-prefixed (either-quoted) "raw" (unescaped) strings
- `f`-prefixed "formatted" strings
  - anything inside braces (`{ ... }`) is evaluated as Python
  - result passed to `str` function for stringification, then concatenated
  - formatted strings can also use format specifiers to

Basics

Strings

- strings can be indexed
  - positive integer: position from left
  - negative integer: position from right
- strings can be "sliced"
  - using `[x:y]` syntax, where x or y could be empty
- strings are immutable

Basics

Strings

```
str1 = "hello"
str2 = 'hello'

str3 = r"Long live metal \m/"

str4 = f"{str1}. I would like to say {str3}"

str5 = """
This is a here-doc. It can span lines.
It will continue going until we run into
the triple-quote again. It is often used
for documentation and simple formatting
purposes.
"""

str6 = "This" "is" "several" "strings"

str7 = str4[6:8]  # "is"
```

Basics

Tuples

- sequences of arbitrary elements, collected
  - think of class with fields
  - but no field names
  - positionally ordered
- destructuring is often done via multiple assignment
- idiomatic/stylistic: use _ for ignored/unused elements

Basics

Tuples

```
ted = "Ted", "Neward", 46
print(ted)                  # ("Ted", "Neward", 46)
charlotte = ("Charlotte", "Neward", 39)
first, last, age = ted
print(first)                # "Ted"
```

Basics

Lists

- ordered sequence of arbitrary (polymorphic) elements
- square-bracket `[...]` surrounded, comma-separated
- mutable, indexable, sliceable
- concatenatable via `+`

Basics

Lists

```
list1 = [1, 2, 3, 4, 5]
list2 = [1, "two", 3.0]
list3 = list1 + list2       # [1, 2, 3, 4, 5, 1, "two", 3.0]
firstlistelement = list1[0] # 1
lastlistelement = list1[-1] # 5
```

Basics

Sets

- unordered sequence of unique arbitrary elements
- initialize by...
  - `{}`-surrounded, comma-separated
  - `set()` (or `frozenset`) function
- supports infix set operations that respect mathematical set operations
  - `x in s`: test `x` for membership in `s`; `not in` is reverse

Basics

Sets

```
set1 = {1, 2, 3, 4, 5}
set2 = {1, 1, 2, 2, 3, 3, 4, 4}
print(set2)                 # "{1, 2, 3, 4}"

set3 = {2, 4, 6}
set4 = {1, 4, 9, 16}
print(set3 & set4)          # "{4}"
print(set3 - set4)          # "{2, 6}"
print(set3 | set4)          # "{16, 1, 2, 4, 6, 9}"
```

Basics

Dictionaries (Hashes)

- unordered sequence of key/value pairs
  - keys are immutable
  - values are mutable
- initialize by...
  - `{}` surrounded, comma-separated

    `key : value` pairs as arguments
  - or `dict()` function

    `key=value` pairs as arguments

Basics

Dictionaries (Hashes)

- access:
  - elements via `[]`-indexing using key
  - `keys()` - sequence of keys
  - `values()` - sequence of values
  - `items()` - sequence of key/value tuples

Basics

Dictionaries (Hashes)

```
dict1 = { "captain":"Antilles" }
dict1["admiral"] = "Ackbar"
k = dict1.keys()
v = dict1.values()
kv = dict1.items()

another_dict = dict(name="Ted",age=47)
print(another_dict)   # {'name':'Ted','age':47}
```

Basics

Python and types

- Python objects always have a type; they are not "untyped"
- Python evaluates all expressions/statements at runtime
- therefore Python is not "untyped" but "runtime type-evaluated and -checked"
- https://docs.python.org/3/library/stdtypes.html

Basics

Python type conversion

- almost any basic type can "convert"/evaluate as a Boolean
  - `None`, 0, and empty collections are all "falsey"
  - anything else is "truthy"
- objects can convert to strings via `str()` function
- objects can define conversion methods

Branching, jumping, iterating, oh my!

FLOW CONTROL

Flow Control

Branching (If)

- `if` condition `:`

  colon is *not optional*

- condition cannot contain assignment
- block is indented; no "scope markers"
- `elif` condition `:`
- `else:`

Flow Control

If

```
age = 45
if age > 50:
    print("You old!")
elif age < 18:
    print("Juvie!")
else:
    print("You not old!")

insult = "Boomer!" if age > 50 else "Millennial!"
```

Flow Control

Stylistic/Idiomatic

- chain `if` conditions for brevity
- avoid placing branch on the same line as the condition
- use `if`/`else` as ternary operator

Flow Control

Pattern-matching (Match) (3.10+)

- `match` value `:`
- followed by `case` condition `:` blocks
  - literal (`case 400:`)
  - several literals (`case 400 | 401 | 402:`)
  - tuple bindings (`case (x, y):`)
  - lists (!)
  - wildcard (`case _:`)
- ... and can be "guarded" by `if` conditions (guard clauses)

Flow Control

Match

```
http_message = ""
match status:
    case 400:
        http_message = "Bad request"
    case 404:
        http_message = "Not found"
    case 418:
        http_message = "I'm a teapot"
    case _:
        http_message = "Something's wrong with the internet"
```

Flow Control

More Match

```
point = (0, 0)
match point:
    case (0, 0):
        print("Origin")
    case (0, y):
        print(f"Y={y}")
    case (x, 0):
        print(f"X={x}")
    case (x, y) if x == y:
        print(f"Y=X at {x}")
    case (x, y):
        print(f"X={x}, Y={y}")
    case _:
        raise ValueError("Not a point")
```

Flow Control

Iteration (While)

- `while` *condition-expr* **:**
- condition cannot contain assignment
- block is indented
- `break` terminates loop
- `continue` immediately restarts loop
- `else:` block executes when condition becomes false

  not executed in case of `break`

Flow Control

While

```
x = 3
while x > 0:
    print("x is still more than 0")
    x = x -1
else:
    print("x is now " + str(x))
```

Flow Control

Iteration (For)

- for can only operate on "iterables"
- `for` *iterator* `in` *iterable* **:**
- block is indented
- `break` terminates, `continue` moves to next
- `else:` block executes when iterable exhausts

  not executed in case of `break`

Flow Control

For

```
for c in range(10):
    print(c)
else:
    print("Can we see c?", c)
print("Can we still see c?", c)

message = "Python is interesting"
for ch in message:
    print(ch)

attendees = ["Marcel", "Roy", "Jeremy"]
for at in attendees:
    print(at)

my_first_tuple = "Ted", "Neward", 47
for t in my_first_tuple:
    print(t)
```

Flow Control

Stylistic/Idiomatic

- use `enumerate()` instead of an index variable
- use `in` to iterate over an iterable

Flow Control

Exception handling

- `raise` object

  throws the object as an exception

- exceptions continue up the stack until handled

  or the program terminates

- `try:` sets up guarded block

  must have one or more `excepts` and/or `finally` clause

Flow Control

Exception handling

- handler clauses evaluate in top-down order
  - `except` : all exceptions
  - `except` type : only exceptions of type or subtype
  - `except` type "as" value : value holds reference to exception instance
  - `except` (type, type, ...) : any of these types
  - `except` (t1, t2, ...) `as` value : any type and hold reference

Flow Control

Exception handling

- `else:` block executed once `try:` completes without error
- `finally:` block *always* executed

Flow Control

Exceptions

```
try:
    bad_div = 1/0
except ZeroDivisionError:
    print("Silly boy; you can't do that")
else:
    print("Else!")
finally:
    print("Finally!")

try:
    print("In a new try block")
    raise Exception()
except Exception:
    print("We just raised this!")
print("Onwards we execute...")
```

Flow Control

Stylistic/Idiomatic

- exceptions are used throughout Python code; follow suit!
  - little to no performance penalty to their use
  - best to follow the convention of the community here
- avoid "swallowing" exceptions with bare `except` clause

Flow Control

With

- `with` expression `as` variable `:`
- expression result must support a "custom context manager"

  this is a convention that assures the expression result will always be cleaned up after use
- most common use is for `open()`-returned file objects

Flow Control

Exceptions

```
with open("data.dat", "w") as datafile:
    datafile.write("Pretend this is data")
# datafile is implicitly close()d after the above block completes
## {{## END with}}
```

Reusable named behavior

FUNCTIONS

Functions

Python comes with many functions

- part of the "builtins" module
- complete list is in Python documentation
- automatically always in top-level scope

  use `dir(__builtins__)` to see complete list

  use `help()` (a builtin function) for documentation

Functions

Some useful builtin functions

- Conversion:

  `bin(),hex(),oct(),bool(),str(),`
  `bytes(),bytearray(),int(),float(),`
  `complex(),chr(),list(),map(),`
  `dict(),set(),tuple()`

- Math operations:

  `abs(),divmod(),pow(),round(),sum()`

- I/O operations:

  `print(),input(),open()`

Functions

Some useful builtin functions

- Iterable functions:

  `all(),any(),filter(),iter(),len(),max(),min(),range(),sorted()`

- Language/platform functions:

  `id(),classmethod(),staticmethod(),property(),type(),breakpoint(),compile(),eval(),exec(),globals(),locals()`

- Full list:

  https://docs.python.org/3/library/functions.html

Functions

Built-in Functions

```
print("Hello, Python!")
print(dir(__builtins__))
```

Functions

User-defined functions

- `def` name `(` parameter(s) `)` **:**
- indent entire body of function
- first line of function is a string -- documentation ("doc-string")
- use `return` to hand back return value
- invoke functions using parens, passing in comma-separated list of arguments

Functions

Defining functions

```
def sayHowdy():
    """This function says howdy and returns the same"""
    print("Howdy!")
    return "I said howdy"

response = sayHowdy()
print(response)             # "I said howdy"
print(sayHowdy.__doc__)     # "This function says..."
```

Functions

Type Hints

- introduced in Python 3.5; documented in PEP 484
  - https://peps.python.org/pep-0484/
- entirely optional, but helpful
- parameters use "Pascal style" colon syntax
- return types use "arrow" syntax
- ***Python will not enforce type hints***
  - IDEs and editors, however, can make use of it to flag type mismatches

Functions

Type-hinting functions

```
def add(lhs : int, rhs : int) -> int:
    return lhs + rhs
def concat(str1 : str, str2 : str) -> str:
    return str1 + str2

print(add(2, 3))            # 5
print(add('1', '2'))        # 12
print(concat(1, 2))         # 3
print(concat('1', '2'))     # 12
```

Functions

Default argument values

- parameter list can have default values assigned
- default values must come after non-defaulted parameters
- note that (non-literal) default values are evaluated at time of definition and only once

Functions

Default argument values

```
def sayGoodMorning(language="english"):
    if (language == "english"):
        return "Good morning!"
    else:
        return f"Sorry, I don't speak {language}"

print(sayGoodMorning())
print(sayGoodMorning("french"))
```

Functions

Keyword arguments

- arguments can be "passed-by-name" at point of call
- allows for reordering of arguments
- positional and keyword can be mixed, if positional arguments come first

Functions

Keyword arguments

```
def saySomething(message="Hello", language="English", times=1):
    """Prints a message in a language a number of times"""
    for x in range(times):
        print(message + " (in " + language + ")")

saySomething(language="German", times=3, message="Good Morning")
```

Functions

Formal parameter collection

- final formal parameter of the form `**name` will be a dictionary instance containing all parameter arguments by name

  name can be anything desired; the two ** are the signal

Functions

Formal parameters

```
def speak(**args):
    print(args)
    for key, value in args.items():
        print("argument " + str(key) + " = " + str(value))

speak(one="1", two="2", three="3")
```

Functions

Arbitrary argument list

- parameter of the form "*name" is a catch-all list for any arguments after
- provides variadic parameter list capabilities

Functions

Aribtrary argument lists

```
def myPrint(prefix, *rest):
    message = prefix
    for r in rest:
        message += " " + r
    print(message)

myPrint("Salutations")
myPrint("Salutations", "planet")
```

Functions

Functions are first-class citizens

- can be captured in variables
- can be passed as arguments
- can be returned from functions

Functions

Function literals

```
fn_lit = saySomething
fn_lit("Hello", "English", 2)
```

Functions

Lambda expressions (function literals)

- syntax:

  `lambda` argument(s) `:` body

  body must be a single expression

- keep lambdas short

  use full function definitions for more than a single expresison or two

  Python will actually enforce brevity here

Functions

Lambdas (aka closures)

```
messages = ["Hello", "Guten morgen", "Bonjour"]
def capIt(x):
    return x.upper()
print(capIt("hello"))   # "HELLO"
up_messages = map(capIt, messages)    # "HELLO", "GUTEN MORGEN", "BONJOUR"
up_messages2 = map(lambda x: x.upper, messages) # same as previous

add_one = lambda x: x+1     # same as def add_one(x): return x+1
result = (lambda x: x+1)(2) # 3
```

Functions

List comprehensions

- succinct way to initialize a sequence via code
- relies heavily on lambda-style syntax
- considered "higher form" among Pythonistas

Functions

Three ways to initialize a list of squares

```
squares1 = []
for x in range(10):
    squares1.append(x**2)
# or...
squares2 = list(map(lambda x: x**2, range(10)))
# or...
squares3 = [x**2 for x in range(10)]
```

Functions

More Comprehensions

```
x = [2, 3, 4, 5, 6]
y = []
for v in x :
    y += [v * 5]        # y == [10, 15, 20, 25, 30]

y2 = map(lambda v : v * 5, x)

y3 = [v * 5 for v in x] # y3 == [10, 15, 20, 25, 30]
```

Functions

More Comprehensions

```
for v in x :
        if v % 2 :
            y += [v * 5]

y2 = map(lambda v : v * 5, filter(lambda u : u % 2, x))

y3 = [v * 5 for v in x if v % 2]
```

Functions

Generators

- define a set of results; do not materialize the list
- lightweight compared to lists
- on demand, each element is generated
- examples:
  - `range()`: function that creates a generator of numbers
  - list comprehension is a generator
  - any function that uses `yield` is a generator

Functions

Generators

```
def generator_function():
    for i in range(10):
        yield i
for i in generator_function():
    print(i)

def fibonacci(n):
    a = b = 1
    for i in range(n):
        yield a
        a,b = b, a+b
```

Functions

Global references and keyword

- Generally, global variables are bad

  but sometimes they're useful and/or necessary
- Global variables have interesting semantics
  - globals can be read from anywhere
  - ... but cannot be modified
  - ... *unless* declared/referenced using the `global` keyword

Functions

Globals

```
player = "Fred"

def doSomething():
    #global player  # without this, it's an error
        # "UnboundLocalError: local variable 'player' referenced before
assignment"
    player = player + " and Jed"
    print(player)

doSomething()
```

Mapping names to objects

NAMESPACES

Namespaces

"Let's do more of those!"

- Python specifically calls out namespaces as a powerful feature
- conceptually no different from a dictionary (`dict`)
- ... but not all namespaces are dictionaries

Namespaces

Examples

- the builtins namespace contains top-level objects loaded at start
- global namespace for a module is created when the module is loaded
- each function invocation has its own namespace (for locals)

Namespaces

A namespaces example

```
def scope_test():
    def do_local():
        spam = "local spam"

    def do_nonlocal():
        nonlocal spam
        spam = "nonlocal spam"

    def do_global():
        global spam
        spam = "global spam"

    spam = "test spam"
    do_local()
    print("After local assignment:", spam)
    do_nonlocal()
    print("After nonlocal assignment:", spam)
    do_global()
    print("After global assignment:", spam)

scope_test()
```

Combining state and behavior, oh my!

CLASSES

Classes

Pythonic classes

- runtime-defined
- runtime-accessible
- implementation inheritable

  all methods dynamically dispatched

Classes

Definition

- `class` name `:`

  prefer CamelCase
- class can contain zero or more "attributes"
  - "data attributes" are fields
  - "function attributes" are methods
- fields are simply introduced
- methods are just functions defined in the class

Classes

Basic syntax

```
class Person:
    """Someday this class will represent a human being"""

    # count is a "class variable" (static)
    count = 0

    # sayHello is a "class method" (static)
    def sayHello():
        print("Hello, world!")

print(Person.count)
Person.sayHello()
```

Classes

Instantiation

- to instantiate, "invoke" the classname
- if an `__init__` method is present, this will initialize the instance
  - takes one or more arguments; first is the instance itself ("self")
  - introduce any instance-specific fields here

Classes

Manipulation

- classes and objects are "open"
  - meaning, we can add members after definition
- use `dir()` on class or instance to see all members
- capturing method reference into variable captures both instance and method
  - execute directly as desired

Classes

Class manipulation

```
intern = Person("Joe", "Intern", 20)
intern.code()
    # "Joe is coding, coding, coding..."

print(dir(intern))
    # (Prints out a lot of methods, most of which we didn't define)

internCodeFunction = intern.code
internCodeFunction()
    # "Joe is coding, coding, coding..."
```

Classes

"Static" vs "instance"

- fields and methods are on class object (singleton/"static")
- instance methods must take explicit "self" parameter
  - first parameter
  - always the value of the object instance when invoked
  - by convention, always called `self`
- fields do not exist unless initialized in

Classes

__init__

```
class Person:
    def __init__(self, fn, ln, a):
        self.firstName = fn
        self.lastName = ln
        self.age = a

    def code(self):
        print(self.firstName + " is coding, coding, coding...")

ted = Person("Ted", "Neward", 48)
ted.code()
```

Classes

Property members

- `property()` function takes getter, setter, and deleter member function
  - optional docstring
- returns an object that "is" the property (which is just a wrapper)
- can also be used as a "decorator" on each member function

Classes

Using property()

```
class Monster:
    def __init__(self, name):
        self._name = name

    def getname(self):
        return self._name

    def setname(self, value):
        self._name = value

    def delname(self):
        del self._name

    name = property(getname, setname, delname, "The Monster's name")

orc = Monster("Orc")
print("The " + orc.name + " attacks you!")
```

Classes

The property decorator

```
class Item:
    def __init__(self, name):
        self._name = name

    @property
    def name(self):
        return self._name

    @name.setter
    def name(self, value):
        self._name = value

    @name.deleter
    def name(self):
        del self._name

magicSword = Item("Sword +1")
print(magicSword.name + " is such a boring name")
magicSword.name = "Sting"  # Much better
```

Classes

Stylistic/Idiomatic

- define `__str__` for human-readable representation
- define `__eq__` and `__hash__` together

Classes

Inheritance

- to inherit, put base class(es) after class name before colon

  `class Student(Person):`

- multiple base classes are permitted

  not common, however

- to override a method, simply define a new one
- to call up to a base class method, use `classname.method()`

Classes

Inheritance

```
class Student(Person):
    def study(self, subject):
        print(self.firstName + " is studying " + subject)
```

Classes

Access Control

- pretty much nothing; everything is public by default
- convention:
  - prefix members with...
    - one underscore for "protected" (client-inaccessible)
    - two underscores for "private" (subclass-inaccessible)
  - the interpreter actually recognizes this convention internally
  - this can help immensely with subclasses

Python's subtle convention for... everything?

MAGIC METHODS

Magic Methods

Names can be awkward

- how can we create "special" methods...
  - ... like "initialization" methods
  - ... or operators
- ... without potentially clashing with programmer-defined names?

Magic Methods

Method conventions

- any "special" method is surrounded by double-underscores

  `__init__`, `__eq__`, `__hash__`, etc
- these are all understood by the interpreter -- do not change semantics
- provides features found in other languages
  - initialization/deinitialization
  - operator overloading
  - comparison

Magic Methods

Object initialization/deinitialization

- `__new__`: Create a new instance
- `__init__`: Initializer for the object
- `__del__`: finalizer/destructor
- `__getattr__`: returns an attribute (field) by name
- `__setattr__`: assigns a value to an attribute (field) by name

Magic Methods

Object comparison operations

- `__lt__`: less-than (`self < other` is True)
- `__gt__`: greater-than (`self > other` is True)
- `__eq__`: equals/equality
- `__ne__`: not-equals/inequality
- `__le__`: less-than-or-equals
- `__ge__`: greater-than-or-equals
- `__hash__`: return an integer "hash" of the

Magic Methods

Magic methods can also add interesting behavior

- `__call__`: makes this object a *callable*
  - in other words, something we can invoke
  - function and method objects are callables
- `__await__`: makes this object an *awaitable*
  - in other words, something that participates in async/await operations
  - function and method `async def` objects are awaitables

Magic Methods

A sample reversing iterator

```
class Reverse:
    """Iterator for looping over a sequence backwards."""
    def __init__(self, data):
        self.data = data
        self.index = len(data)

    def __iter__(self):
        return self

    def __next__(self):
        if self.index == 0:
            raise StopIteration
        self.index = self.index - 1
        return self.data[self.index]
```

Using Python as an application scripting langauge

EMBEDDING CPYTHON

Embedding CPython

CPython was designed to be embedded

- That is, loaded into other applications
- C-based interpreter API
  - This means anything that can call C can host Python
  - ... which means pretty much anything can host Python

Embedding CPython

Hosting Python

```
#define PY_SSIZE_T_CLEAN
#include <Python.h>

int
main(int argc, char *argv[])
{
    wchar_t *program = Py_DecodeLocale(argv[0], NULL);
    if (program == NULL) {
        fprintf(stderr, "Fatal error: cannot decode argv[0]\n");
        exit(1);
    }
    Py_SetProgramName(program);  /* deprecated in 3.11, but hey still common */
    Py_Initialize();
    PyRun_SimpleString("from time import time,ctime\n"
                       "print('Today is', ctime(time()))\n");
    if (Py_FinalizeEx() < 0) {
        exit(120);
    }
    PyMem_RawFree(program);
    return 0;
}
```

Embedding CPython

C Compiling and linking

- requires Python include and lib directories
  - the location of these will vary with your installation
- Python has your back on this:
  - `import sysconfig`
  - `print(sysconfig.get_config_vars())`

Embedding CPython

Python has also been written in other languages

- For example...
  - Jython is a version written for Java/JVM
  - GraalPy is a Python VM written on top of GraalVM (JVM)
  - IronPython is a version for C#/.NET
  - Pyodide ia a Python VM in WebAssembly
- These are *not* official CPython interpreters

  though they are often based on CPython source

Where to go for more

RESOURCES

Online

Websites

- Python Tutorial

  https://docs.python.org/3/tutorial/index.html

- Python Language Reference

  https://docs.python.org/3/reference/index.html

- Python Library Reference

  https://docs.python.org/3/library/index.html

- Real Python website

  https://realpython.com

Online

Deeper Python language

- "A Guide to Python's Magic Methods"

  https://github.com/RafeKettler/magicmethods/blob/master/magicmethods.pdf

- "Writing Idiomatic Python3"

  Jeff Knupp

- "Effective Python"

  Slatkin Brett (Addison-Wesley)

Credentials

Who is this guy?

- Architect, Engineering Manager/Leader, "force multiplier"
  - http://www.newardassociates.com
  - http://blogs.newardassociates.com
- Sr Distinguished Engineer, Capital One
- Educative (http://educative.io) Author
  - *Performance Management for Engineering Managers*
- Books